

Java Backend Interview Series

Sub-Chapter 1 12

JVM Internals

ClassLoader . Memory . GC . JIT . Bytecode . Object Layout

Motivation + Before/After Code + Import Blocks + 30 Q&A + Pitfalls + Tips

1 JVM Architecture and Runtime Data Areas

1.1 Introduction

The JVM is a stack-based virtual machine that executes Java bytecode. It has three main subsystems: ClassLoader (loads .class files), Runtime Data Areas (memory partitions), and Execution Engine (interprets/compiles bytecode). Understanding these areas is essential for diagnosing OOM errors, tuning GC, and debugging memory leaks.

1.2 Runtime Data Areas

Memory Area	Per-Thread?	Contents	OOM Error Type
Heap	No (shared)	Objects, arrays, instance data	java.lang.OutOfMemoryError: Java heap space
Method Area / Metaspace	No (shared)	Class metadata, static fields, constants	OutOfMemoryError: Metaspace
JVM Stack	Yes	Stack frames: local vars, operand stack	StackOverflowError
PC Register	Yes	Address of current bytecode instruction	Never OOM
Native Stack	Yes	Native (JNI) method call frames	StackOverflowError (native)
Code Cache	No (shared)	JIT-compiled native code	OOM: CodeCache is full
Direct Memory	No (shared)	Off-heap NIO ByteBuffer allocations	OOM: Direct buffer memory

1.3 Monitoring Heap and Metaspace

```
// Heap monitoring via Runtime

Runtime rt = Runtime.getRuntime();

long maxHeap = rt.maxMemory(); // -Xmx setting

long totalHeap = rt.totalMemory(); // current committed heap

long freeHeap = rt.freeMemory(); // free in committed heap

long usedHeap = totalHeap - freeHeap;

System.out.printf("Used: %d MB / Max: %d MB\n",
    usedHeap/1024/1024, maxHeap/1024/1024);

// Key JVM flags

// -Xms512m -- initial heap size

// -Xmx2g -- maximum heap size

// -Xss512k -- per-thread stack size (default ~512k-1m)

// -XX:MetaspaceSize=128m -- initial Metaspace trigger
```

```
// -XX:MaxMetaspaceSize=256m -- cap Metaspace (default: unlimited!)

// -XX:+UseCompressedOops -- compress 64-bit object pointers to 32-bit

// -XX:+PrintGCDetails -Xloggc:gc.log -- GC logging
```

Runtime heap monitoring and key JVM flags

WARNING: MaxMetaspaceSize defaults to unlimited. If classes are generated dynamically (e.g., JRuby, Groovy, heavy reflection) and old classloaders are not GC'd, Metaspace grows unboundedly. Set -XX:MaxMetaspaceSize in production.

2 ClassLoader Hierarchy and Delegation

2.1 Introduction

ClassLoaders load .class files into the JVM. They form a parent-delegation hierarchy: when asked to load a class, a ClassLoader first delegates to its parent. Only if the parent cannot find it does the child attempt its own search. This ensures core JDK classes (java.lang.Object etc.) always come from Bootstrap, not user code.

2.2 The Three Standard ClassLoaders

ClassLoader	Loads	Parent	Implementation
Bootstrap	java.lang.*, java.util.*, JDK core classes	None (root)	Native C++ code (not a Java class)
Platform (Java 9+)	java.sql, javax.*, named module classes	Bootstrap	jdk.internal.loader.ClassLoaders\$PlatformClassLoader
Application / App	Classes on -classpath / --module-path	Platform	jdk.internal.loader.ClassLoaders\$AppClassLoader
Custom	Anything you define (plugins, hot-reload)	Usually App	Extend ClassLoader or URLClassLoader

2.3 Custom ClassLoader -- Loading from Bytes

```
import java.util.Map;
import java.nio.file.Files;
import java.nio.file.Path;

public class ByteArrayClassLoader extends ClassLoader {
    private final Map<String, byte[]> classBytesMap;

    public ByteArrayClassLoader(Map<String, byte[]> classBytesMap) {
        super(ByteArrayClassLoader.class.getClassLoader()); // parent = App CL
        this.classBytesMap = classBytesMap;
    }

    @Override
    protected Class<?> findClass(String name) throws ClassNotFoundException {
        byte[] bytes = classBytesMap.get(name);

        if (bytes == null) throw new ClassNotFoundException(name);

        // defineClass validates bytecode and creates a Class object
        return defineClass(name, bytes, 0, bytes.length);
    }
}

// Usage: load plugin from byte array
```

```
byte[] pluginBytes = Files.readAllBytes(Path.of("Plugin.class"));

ByteArrayClassLoader cl = new ByteArrayClassLoader(
    Map.of("com.example.Plugin", pluginBytes));

Class<?> pluginClass = cl.loadClass("com.example.Plugin");

Object plugin = pluginClass.getDeclaredConstructor().newInstance();

// Class.forName vs ClassLoader.loadClass
// Class.forName: loads AND initializes (runs static blocks)
// ClassLoader.loadClass: loads but does NOT initialize
```

Custom ClassLoader: loading class from byte array

WARNING: ClassLoader leaks: if a ClassLoader is referenced by a live object (e.g., ThreadLocal, static field in a loaded class), it and ALL its loaded classes cannot be GC'd. This causes Metaspace OOM in application servers on re-deploy.

3 JVM Memory Areas -- Heap Generations

3.1 Introduction

The heap is divided into generations based on the generational hypothesis: most objects die young. Young generation collects frequently and cheaply. Old (Tenured) generation collects rarely and expensively. Metaspace (Java 8+) replaced PermGen for class metadata.

3.2 Heap Structure

Region	Abbreviation	Contents	Collection
Eden	Young Gen	New objects first allocated here (TLAB)	Minor GC (frequent, fast)
Survivor 0 (S0)	Young Gen	Survived one or more minor GCs	Minor GC
Survivor 1 (S1)	Young Gen	Alternate survivor space (copy target)	Minor GC
Old / Tenured	Old Gen	Objects surviving many minor GCs (age threshold)	Major / Full GC (rare, slow)
Metaspace	Off-heap	Class metadata, method bytecode, interned strings	GC on class unloading

3.3 TLAB and Object Allocation

```
// TLAB: Thread Local Allocation Buffer

// Each thread has its own slice of Eden -- avoids contention on allocation

// new Object() is typically just a pointer bump inside TLAB -- very fast

// Key flags:

// -XX:+UseTLAB -- enable TLAB (default: true)

// -XX:TLABSize=128k -- per-thread buffer size

// -XX:+PrintTLAB -- log TLAB usage

// Large Object allocation (bypasses Young Gen)

// -XX:G1HeapRegionSize=1m -- G1 region size

// Objects > G1HeapRegionSize/2 go directly to Old Gen as Humongous objects

// Object age threshold for promotion

// -XX:MaxTenuringThreshold=15 -- max minor GC survivals before promotion

// Inspect heap from code

Runtime rt = Runtime.getRuntime();

System.out.printf("Heap used: %d MB%n",
```

```
(rt.totalMemory() - rt.freeMemory()) / 1024 / 1024);
```

TLAB, large object allocation, and tenuring flags

3.4 PermGen vs Metaspace

Aspect	PermGen (Java <= 7)	Metaspace (Java 8+)
Location	Heap (fixed max size)	Native memory (off-heap)
Default max	Fixed (e.g., 64-256 MB)	Unlimited (OS memory limit)
Typical OOM	OOM: PermGen space	OOM: Metaspace (if MaxMetaspaceSize set)
Interned Strings	Stored in PermGen	Moved to Heap (Java 7+)
Tuning flag	-XX:MaxPermSize=256m	-XX:MaxMetaspaceSize=256m

4 Garbage Collection Algorithms

4.1 Introduction

Java GC automatically reclaims heap memory used by unreachable objects. Different collectors trade off throughput (application time) against pause latency (stop-the-world time). Java 9+ uses G1 as the default; Java 15+ offers ZGC and Shenandoah for sub-millisecond pauses.

4.2 GC Fundamentals

Algorithm	How It Works	Cost
Mark-and-Sweep	Mark reachable objects; sweep unmarked ones	Fragmentation (holes in heap)
Mark-Compact	Mark reachable; compact survivors to one end	No fragmentation; slower (moves objects)
Copying (Young Gen)	Copy live objects from Eden+S0 to S1; flip roles	Fast (short-lived objects); needs double space
Generational	Apply copying to Young Gen (frequent); mark-compact to Old Gen (rare)	Best of both based on generational hypothesis

4.3 GC Collectors Comparison

Collector	Java Version	Pause	Strengths	Use Case
Serial	All	High (STW)	Simple, low overhead	Single-core, small heaps
Parallel (Throughput)	All	Medium (STW)	Max throughput	Batch jobs, non-latency-sensitive
CMS	<=Java 9	Low (mostly concurrent)	Low pauses	Deprecated in Java 9, removed in Java 14
G1	Java 9+ (default)	Predictable (soft target)	Balanced; region-based	General purpose, heap >= 4 GB
ZGC	Java 15+	Sub-millisecond	Minimal pause regardless of heap size	Large heap, latency-critical services
Shenandoah	Java 15+	Sub-millisecond	Concurrent compaction	Low-latency; alternative to ZGC

4.4 G1 GC -- Region-Based Collector

```
// G1 divides heap into equal-sized regions (1-32 MB each)

// No fixed Young/Old boundaries -- regions can be any generation

// Garbage-first: collects regions with most garbage first

// Key G1 flags

// -XX:+UseG1GC -- enable G1 (default Java 9+)

// -XX:MaxGCPauseMillis=200 -- soft pause target (default 200ms)
```

```
// -XX:G1HeapRegionSize=8m -- region size (power of 2, 1-32 MB)

// -XX:G1NewSizePercent=5 -- min young gen %

// -XX:G1MaxNewSizePercent=60 -- max young gen %


// G1 GC log analysis (Java 11+ -Xlog:gc*)

// [GC pause (G1 Evacuation Pause) (young) 450M->120M(512M) 15ms] -- minor

// [GC pause (G1 Mixed) 450M->200M(512M) 25ms] -- mixed (young+old)


// Humongous objects: > G1HeapRegionSize/2

// Allocated directly in old-gen humongous regions

// Can cause GC pressure -- avoid frequent large object creation


// ZGC flags

// -XX:+UseZGC -- sub-millisecond pauses

// -XX:+ZGenerational -- generational ZGC (Java 21, default Java 23)
```

G1 GC flags and log interpretation

TIP: Start with `-XX:MaxGCPauseMillis=200` (G1 default). Monitor GC logs. If pause targets are missed, consider `-XX:+UseZGC` for services requiring < 10ms pauses. Do not tune GC blindly -- always measure with production-like workloads.

5 JIT Compilation -- From Bytecode to Native

5.1 Introduction

The JVM starts by interpreting bytecode. Hot methods (called frequently) are JIT-compiled to native machine code by the C1 (client) or C2 (server) compiler. Tiered compilation (Java 8+) uses both: C1 for fast startup, C2 for peak throughput.

5.2 Tiered Compilation Levels

Level	Mode	When	Quality
0	Interpreter	All methods initially	Slowest; collects profiling data
1	C1 -- simple	After ~100 invocations	Fast compile; no profiling
2	C1 -- limited profiling	Quickly profiled methods	Fast compile; some profiling
3	C1 -- full profiling	Hot methods	Profile-based optimization data
4	C2 -- full optimization	Very hot methods (10,000+ invocations)	Maximum speed; slow compile

5.3 Key JIT Optimizations

```
// 1. Method Inlining -- most impactful optimization

// Replaces a call to a small method with the method body itself

int add(int a, int b) { return a + b; } // <= 35 bytecodes -> gets inlined

// int result = obj.add(3, 4);

// JIT transforms to: int result = 3 + 4; (no call overhead)

// Control inlining:

// -XX:MaxInlineSize=35 -- max bytecodes for trivial inlining

// -XX:FreqInlineSize=325 -- max bytecodes for hot method inlining

// -XX:+PrintInlining -- log what gets inlined


// 2. Escape Analysis -- object on stack instead of heap

// If an object doesn't escape its creating method, JIT puts it on the stack

void process() {

    Point p = new Point(1, 2); // p does not escape this method

    int dist = p.x + p.y; // JIT may eliminate the allocation entirely!

} // p is scalar-replaced -- no heap allocation, no GC pressure
```

```
// 3. On-Stack Replacement (OSR)

// Replaces an ALREADY-RUNNING interpreted loop with compiled version

// Allows long-running loops to benefit from JIT without method re-entry


// 4. Deoptimization

// If a JIT assumption is violated (e.g., a "final" virtual call becomes polymorphic)

// JIT reverts to interpreter for that method

// -XX:+PrintDeoptimization -- log deoptimizations
```

JIT optimizations: inlining, escape analysis, OSR, deoptimization

TIP: Escape analysis means short-lived objects in tight loops may have ZERO GC cost -- the JIT puts them on the stack. Avoid premature optimization of allocations until profiling shows GC pressure.

6 Object Memory Layout

6.1 Introduction

Every Java object on the heap has a header followed by its fields. Understanding object layout helps explain GC overhead, cache-line efficiency, false sharing, and why small objects still consume significant memory.

6.2 Object Header Structure

Part	Size (64-bit +CompressedOops)	Contents
Mark Word	8 bytes	Hash code, GC age (4 bits), lock state (2 bits), biased-lock thread ID
Class Pointer (Klass)	4 bytes (compressed) / 8 bytes	Pointer to class metadata in Metaspace
Array Length	4 bytes (arrays only)	Number of array elements
Instance Fields	Varies	Field values in JVM-determined order (not declaration order)
Alignment Padding	0-7 bytes	Pad to next 8-byte boundary (JVM objects are 8-byte aligned)

6.3 JOL -- Java Object Layout Tool

```
import org.openjdk.jol.info.ClassLayout;
import org.openjdk.jol.info.GraphLayout;

// Add to pom.xml:

// <dependency>
//   <groupId>org.openjdk.jol</groupId>
//   <artifactId>jol-core</artifactId>
//   <version>0.17</version>
// </dependency>

import org.openjdk.jol.info.ClassLayout;

System.out.println(ClassLayout.parseClass(Integer.class).toPrintable());

// java.lang.Integer object internals:
// OFFSET SIZE TYPE DESCRIPTION
// 0 4 (object header: mark)
// 4 4 (object header: mark)
// 8 4 (object header: class)
// 12 4 int Integer.value
// Instance size: 16 bytes
```

```
// An empty object (new Object()) is 16 bytes (header only)
// A boolean field still occupies 4 bytes due to alignment!

// GraphLayout shows object graph size
import org.openjdk.jol.info.GraphLayout;

System.out.println(GraphLayout.parseInstance(new int[100]).toFootprint());

// shows actual memory footprint including referenced objects
```

JOL: inspecting object layout and size

WARNING: Field order in the JVM is NOT the same as declaration order. The JVM reorders fields to minimize padding (longs/doubles first, then ints, then shorts/chars, then bytes/booleans, then object references). This affects cache-line alignment calculations.

7 GC Tuning and Memory Leak Detection

7.1 GC Tuning Flags

```
# Heap sizing

-Xms2g -Xmx2g # set initial = max to avoid heap resizing pauses

-Xmn512m # young gen size (G1 manages this automatically)

# G1 tuning

-XX:+UseG1GC

-XX:MaxGCPauseMillis=100 # pause time target (soft goal)

-XX:G1HeapRegionSize=4m # increase for large heaps

-XX:G1ReservePercent=10 # empty reserve to reduce evacuation failures

# ZGC (Java 15+)

-XX:+UseZGC # enable ZGC

-Xmx16g # ZGC works best with >= 8 GB heap

# GC logging (Java 11+ unified logging)

-Xlog:gc*:file=gc.log:time,uptime,level,tags:filecount=10,filesize=20m

# Diagnostic

-XX:+HeapDumpOnOutOfMemoryError # dump on OOM

-XX:HeapDumpPath=/tmp/heap.hprof

-XX:+ExitOnOutOfMemoryError # restart pod on OOM (Kubernetes)
```

GC tuning flags reference

7.2 Common Memory Leak Causes

Leak Pattern	Root Cause	Fix
Static collections	Static Map/List accumulates entries, never cleared	Use size limit, TTL eviction, or weak references
ThreadLocal leak	Thread pool reuses threads; ThreadLocal value never removed	Call ThreadLocal.remove() in finally block
Inner class reference	Non-static inner class holds implicit reference to outer class	Use static nested class or extract to separate class
Unclosed streams/resources	InputStream, Connection, etc. not closed	Use try-with-resources

Leak Pattern	Root Cause	Fix
ClassLoader leak	Loaded class holds reference to its ClassLoader	Null out ClassLoader references; use OSGi or module system
Event listeners	Added but never removed (observer pattern)	Remove listeners on unregistration/shutdown

7.3 ThreadLocal Leak Pattern and Fix

```
import java.lang.ScopedValue;

// LEAK: ThreadLocal value not removed in thread pool

static final ThreadLocal<Connection> connHolder = new ThreadLocal<>();

void processRequest() {
    connHolder.set(openConnection());
    doWork();
    // FORGOT connHolder.remove() -- connection and any referenced objects
    // live as long as the thread (indefinitely in a pool)
}

// FIX: always remove in finally
void processRequestSafe() {
    connHolder.set(openConnection());
    try {
        doWork();
    } finally {
        connHolder.remove(); // guaranteed cleanup
    }
}

// BETTER (Java 21): use ScopedValue -- auto-cleaned, immutable
static final ScopedValue<Connection> CONN = ScopedValue.newInstance();
ScopedValue.where(CONN, openConnection()).run(() -> doWork());
// Connection goes out of scope automatically when run() returns
```

ThreadLocal memory leak and fix with ScopedValue

WARNING: ThreadLocal is one of the most common memory leak sources in Java applications. In thread pool environments (Tomcat, Spring, Netty), always call remove() in a finally block. Prefer ScopedValue (Java 21) for new code.

8 Class File Format and Bytecode

8.1 Introduction

A .class file is a binary format beginning with magic number 0xCAFEBAE. The JVM validates this format on load. Understanding bytecode helps with debugging, performance analysis, and advanced framework development (bytecode instrumentation, AspectJ, cglib).

8.2 Class File Structure

Section	Contents
Magic + Version	0xCAFEBAE; major/minor version (Java 21 = 65)
Constant Pool	String literals, class refs, method refs, field refs (UTF-8 strings, integers)
Access Flags	public, final, abstract, interface, enum, record, module (bit mask)
This/Super/Interfaces	Index into constant pool for class name, superclass, implemented interfaces
Fields	Name, type descriptor, access flags, optional ConstantValue attribute
Methods	Name, signature descriptor, Code attribute (bytecode, max_stack, max_locals, exception table)
Attributes	SourceFile, LineNumberTable, LocalVariableTable, StackMapTable, InnerClasses

8.3 Key Bytecode Instructions

```
// Disassemble with: javap -c -verbose MyClass.class

// Java source

public void greet(String name) {

    System.out.println("Hello, " + name);

}

// Bytecode (javap -c)

// 0: getstatic #2 // Field java/lang/System.out:Ljava/io/PrintStream;

// 3: new #3 // class java/lang/StringBuilder

// 6: dup

// 7: invokespecial #4 // Method java/lang/StringBuilder."<init>":()V

// 10: ldc #5 // String "Hello, "

// 12: invokevirtual #6 // Method StringBuilder.append:(Ljava/lang/String;)Ljava/lang/StringBuilder;

// 15: aload_1

// 16: invokevirtual #6 // Method StringBuilder.append

// 19: invokevirtual #7 // Method StringBuilder.toString:()Ljava/lang/String;
```

```
// 22: invokevirtual #8 // Method PrintStream.println:(Ljava/lang/String;)V

// 25: return

// invoke opcodes:

// invokevirtual -- instance method (runtime dispatch on receiver type)

// invokeinterface -- interface method (also runtime dispatch)

// invokespecial -- private, constructor, super (compile-time dispatch)

// invokestatic -- static method

// invokedynamic -- dynamic dispatch (lambdas, method handles)
```

Bytecode disassembly and invoke opcodes

TIP: Use `javap -c -verbose ClassName.class` to inspect bytecode. Understanding invoke opcodes explains why interface dispatch can be slower than class dispatch, and why `invokedynamic` enables efficient lambdas.

9 Native Memory and Off-Heap Buffers

9.1 Introduction

Java can allocate memory outside the GC-managed heap (off-heap). This is useful for large caches, network I/O (NIO), and avoiding GC pressure. Direct ByteBuffers allocate off-heap; the Java 22 Foreign Function and Memory API (FFM) provides a safe, structured off-heap access API.

9.2 Direct vs Heap ByteBuffer

```
import java.nio.ByteBuffer;
import java.nio.MappedByteBuffer;
import java.nio.channels.FileChannel;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;

// Heap ByteBuffer -- backed by a Java byte[] on the heap
ByteBuffer heapBuf = ByteBuffer.allocate(1024);

heapBuf.putInt(42);

heapBuf.flip();

int val = heapBuf.getInt();

// Direct ByteBuffer -- backed by native memory (off-heap)
ByteBuffer directBuf = ByteBuffer.allocateDirect(1024 * 1024); // 1 MB off-heap
directBuf.putInt(99);

directBuf.flip();

int val2 = directBuf.getInt();

// Direct buffers are faster for I/O (no copy to native before syscall)
// Managed by PhantomReference -- freed when Cleaner runs
// NOT freed by GC directly -- can cause OOM: Direct buffer memory

// Total direct memory: -XX:MaxDirectMemorySize=512m (default = -Xmx)
// Monitor: ManagementFactory.getMemoryMXBean() or JMX

// MappedByteBuffer -- memory-mapped file
try (FileChannel fc = FileChannel.open(Path.of("data.bin"), StandardOpenOption.READ)) {
    MappedByteBuffer mmap = fc.map(FileChannel.MapMode.READ_ONLY, 0, fc.size());

    // OS pages file into memory on access -- no explicit read call needed
}
```

Direct ByteBuffer vs heap ByteBuffer, MappedByteBuffer

WARNING: Direct ByteBuffers are NOT freed immediately when the Java reference goes out of scope. They are freed by a Cleaner/PhantomReference when GC collects the ByteBuffer object. Under allocation pressure, direct memory can OOM before heap does. Set -XX:MaxDirectMemorySize appropriately.

10 Interview Q and A -- JVM Internals

30 questions covering standard and advanced depth. [ADV] marks senior-level questions.

Q: What are the main runtime data areas of the JVM?

A: Heap (shared, objects/arrays), Method Area/Metaspace (shared, class metadata/static fields), JVM Stack (per-thread, frames), PC Register (per-thread, current instruction address), Native Method Stack (per-thread, JNI frames), Code Cache (shared, JIT output), Direct Memory (off-heap NIO).

Q: What is the difference between PermGen and Metaspace?

A: PermGen (Java <=7): fixed-size heap region for class metadata, default max ~64-256MB, caused OOM: PermGen space on re-deploys. Metaspace (Java 8+): uses native memory (off-heap), unlimited by default, grows dynamically. Set -XX:MaxMetaspaceSize to cap it.

Q: What is a ClassLoader and what is parent delegation?

A: A ClassLoader loads .class files into the JVM. Parent delegation: when asked to load a class, it first delegates to its parent. Only if the parent cannot find it does the child attempt its own lookup. This ensures JDK core classes always come from Bootstrap, preventing user code from shadowing java.lang.Object.

Q: What are the three standard ClassLoaders?

A: Bootstrap (native, loads rt.jar/java.* in Java 9+ jdk-modules), Platform (Java 9+, loads javax.*, java.sql etc.), Application (loads classpath/module-path classes). Custom ClassLoaders extend one of these.

Q: What is the difference between Class.forName() and ClassLoader.loadClass()?

A: Class.forName(name) loads AND initializes the class (runs static initializer blocks). ClassLoader.loadClass(name) loads but does NOT initialize. Use Class.forName for JDBC driver loading (needs static block to register). Use loadClass when you want to inspect the class without running its static code.

Q: What is the Young Generation and how does Minor GC work?

A: Young Gen = Eden + Survivor0 + Survivor1. New objects are allocated in Eden (via TLAB). Minor GC: copies live objects from Eden+S0 to S1, increments their age, flips S0 and S1. Objects older than MaxTenuringThreshold (default 15) are promoted to Old Gen. Minor GC is fast because it only processes Young Gen.

Q: What is a TLAB?

A: Thread Local Allocation Buffer: each thread has a private slice of Eden. Object allocation (new) is just a pointer bump within TLAB -- no locking or synchronization needed. When TLAB is exhausted, a new one is requested from Eden. This makes allocation very fast.

Q: What does G1 GC do differently from the parallel collector?

A: G1 divides the heap into equal-sized regions (1-32 MB) rather than a fixed Young/Old layout. It prioritizes collecting the regions with the most garbage first (Garbage First). It has a configurable pause target (MaxGCPauseMillis) and uses concurrent marking to minimize stop-the-world time.

Q: What is a Humongous object in G1 GC?

A: An object larger than half a G1 region size. It is allocated directly in consecutive old-gen regions (Humongous regions). Humongous allocations bypass TLAB and Young Gen, trigger concurrent marking immediately, and can cause GC pressure. Avoid frequently allocating very large objects.

Q: What is ZGC?

A: A low-latency GC (Java 15+) designed for sub-millisecond pause times regardless of heap size. It performs concurrent marking and compaction -- most work happens while the application runs. Trade-off: slightly more CPU overhead than G1 for the same throughput.

Q: What is JIT compilation?

A: Just-In-Time compilation: the JVM starts interpreting bytecode. Methods called frequently (hot) are compiled to native machine code by C1 (fast, basic optimization) then C2 (slower, aggressive optimization). Tiered compilation uses both: C1 for startup, C2 for peak throughput.

Q: What is method inlining?

A: JIT replaces a method call with the method body at the call site. For small methods (default: ≤ 35 bytecodes), this eliminates call overhead, enables further optimizations (dead code elimination, loop fusion), and reduces stack frame churn. It is typically the highest-impact JIT optimization.

Q: What is escape analysis?

A: JIT analysis determining if an object 'escapes' its creating method (assigned to a field, passed to another thread, returned, etc.). Non-escaping objects can be stack-allocated or scalar-replaced -- no heap allocation, no GC overhead. `new Point(1,2)` in a tight loop with no escape may have zero allocation cost.

Q: What is a Java object's memory layout?

A: Header: mark word (8 bytes: hash, GC age, lock state) + class pointer (4 bytes compressed/8 bytes uncompressed). Then instance fields (ordered by JVM for minimal padding). Arrays have an additional 4-byte length field. Total size rounded up to 8-byte alignment. Minimum object size: 16 bytes.

Q: What information is in the mark word of an object header?

A: Identity hash code (when computed), GC age (4 bits for tenuring threshold), lock state (2 bits: unlocked/biased/lightweight/heavyweight), and biased-lock thread ID (when biased locking is active). The bits are reused for different purposes (hash vs lock) based on the lock state field.

Q: What causes a ClassLoader memory leak?

A: If a class loaded by a ClassLoader holds a reference (via static field, ThreadLocal, or registered listener) that keeps the ClassLoader reachable after the app is undeployed, neither the ClassLoader nor any of its loaded classes can be GC'd. This fills Metaspace on repeated red deploys in application servers.

Q: What are common Java memory leak patterns?

A: 1) Static collections that grow without eviction. 2) ThreadLocal values not removed in thread pools. 3) Non-static inner classes holding outer class reference. 4) Unclosed streams/connections (use try-with-resources). 5) Event listeners added but never removed. 6) ClassLoader leaks on re-deploy.

Q: What is the difference between heap ByteBuffer and direct ByteBuffer?

A: Heap ByteBuffer: backed by `byte[]` on the Java heap. Easy to use; GC-managed; requires an extra copy to native memory before I/O syscalls. Direct ByteBuffer: backed by native memory. Faster for I/O (no copy needed); not GC-managed (freed by Cleaner when buffer is collected); can cause OOM: Direct buffer memory.

Q: What is the magic number of a Java .class file?

A: 0xCAFE BABE. The JVM verifies this 4-byte sequence at the start of every .class file before loading. An invalid magic number causes `ClassFormatError`.

Q: What is the difference between invokevirtual and invokeinterface?

A: Both dispatch based on the runtime receiver type. `invokevirtual` is for class methods -- the JVM can use vtable dispatch (fast, offset is fixed at compile time). `invokeinterface` is for interface methods -- dispatch requires searching the itable (interface method table) because a class can implement multiple interfaces, making the offset variable.

Q: [ADV] Explain how escape analysis enables scalar replacement.

A: If a non-escaping object is analyzed and its fields are used but the object itself is never treated as a whole (not passed by reference), the JIT can decompose (scalar-replace) it into individual local variables of primitive types. This eliminates the heap allocation entirely -- the object 'fields' live in CPU registers or on the stack.

Q: [ADV] What is On-Stack Replacement (OSR) in JIT compilation?

A: OSR replaces an ALREADY-EXECUTING interpreted method (typically a long-running loop) with the JIT-compiled version -- mid-execution. Without OSR, a long loop would finish in interpreted mode even if the

method was compiled. OSR transfers the interpreted state (local variables, operand stack) to the compiled frame.

Q: [ADV] What is deoptimization and when does the JVM perform it?

A: Deoptimization discards a JIT-compiled version and falls back to interpretation, typically when a JIT assumption is violated. Example: JIT compiled a virtual call assuming only one implementation exists (monomorphic); a new subclass is loaded invalidating that assumption. The compiled code is abandoned; the method reverts to interpreted mode until re-profiled.

Q: [ADV] How does G1 mixed GC work?

A: After concurrent marking identifies live objects in Old Gen regions, G1 schedules mixed GC cycles that collect both Young Gen regions AND selected Old Gen regions (those with most garbage). This incrementally cleans Old Gen with bounded pauses, avoiding a single long full GC. Mixed GC continues until Old Gen occupancy drops below G1HeapWastePercent.

Q: [ADV] What is the difference between Minor GC, Major GC, and Full GC?

A: Minor GC: collects Young Gen only -- fast (copying). Major GC: collects Old Gen (used loosely, often means Old Gen concurrent marking). Full GC: collects ALL generations (Young + Old + Metaspace) -- stop-the-world, expensive. Full GC is triggered by promotion failure or explicit System.gc(). Avoid Full GC in production.

Q: [ADV] Explain what happens internally when you call new Object() in a tight loop.

A: 1) JIT compiles the loop. 2) Escape analysis determines the object doesn't escape. 3) Scalar replacement eliminates the allocation (object 'fields' go to registers). If the object does escape: 4) Thread reads its TLAB pointer. 5) Checks if space fits (TLAB bump pointer). 6) If yes: pointer bump (2 ns). 7) If TLAB exhausted: slow path allocates new TLAB from Eden.

Q: [ADV] What is the remembered set in G1 GC?

A: Each G1 region maintains a Remembered Set (RSet) -- a set of cards (512-byte heap regions) in other regions that contain references into this region. During Minor GC, RSets allow G1 to process only Young Gen without scanning the entire Old Gen for cross-region references. Maintaining RSets has CPU and memory overhead.

Q: [ADV] How do class file major version numbers map to Java versions?

A: Major version = Java version + 44 (for Java 8+). Java 8 = 52, Java 11 = 55, Java 17 = 61, Java 21 = 65. If you compile with Java 21 and run on Java 17 JVM, you get `UnsupportedClassVersionError: class file version 65 but max supported 61`. --release flag in javac ensures bytecode compatibility.

Q: [ADV] Explain the Compressed OOPs optimization.

A: On 64-bit JVMs with heap <= 32 GB, -XX:+UseCompressedOops (default) compresses 64-bit object pointers to 32 bits by encoding them as heap-relative offsets (shifted by 3 bits, leveraging 8-byte alignment). This saves 4 bytes per reference field and keeps cache lines fuller. Disabling it increases reference sizes and reduces cache efficiency.

Q: [ADV] What is the Code Cache and what happens when it fills?

A: The Code Cache is a native memory region storing JIT-compiled code. -XX:ReservedCodeCacheSize (default 240 MB) controls its max. If full, the JVM stops JIT-compiling -- all new methods run interpreted. You see 'CodeCache is full' warning. JIT de-optimizes rarely-used compiled methods to reclaim space. Fix: increase -XX:ReservedCodeCacheSize or reduce the number of loaded classes.

11 Summary -- JVM Internals Cheat Sheet

Topic	Key Facts	Common Interview Pitfall
Memory Areas	Heap, Metaspace, Stack (per-thread), Code Cache, Direct	PermGen gone since Java 8 -- it is Metaspace now
ClassLoader	Bootstrap->Platform->App; parent delegation; define = load+link+init	forName() runs static blocks; loadClass() does not
Young Gen	Eden+S0+S1; TLAB bump allocation; Minor GC copies live objects	Objects exceeding MaxTenuringThreshold promote to Old Gen
GC Collectors	Serial<Parallel<G1(default)<ZGC; pause vs throughput trade-off	CMS is deprecated/removed; G1 is the Java 9+ default
G1 specifics	Region-based; MaxGCPauseMillis target; Humongous objects > half-region	Mixed GC = Young + selected Old Gen regions
JIT	Tiered: interpreter -> C1 -> C2; inlining; escape analysis; deoptimization	Deoptimization happens when JIT assumptions are violated
Object Layout	Header (mark + klass) + fields (JVM-ordered) + padding to 8 bytes	Smallest object is 16 bytes (not 0)
Memory Leaks	Static collections, ThreadLocal, inner class refs, ClassLoader leaks	ThreadLocal.remove() must be in finally in thread pools
Bytecode	0xCAFEBADE magic; constant pool; invoke opcodes; invokedynamic for lambdas	invokeinterface needs itable scan; invokevirtual uses vtable
Off-Heap	DirectByteBuffer, MappedByteBuffer; -XX:MaxDirectMemorySize	Direct memory NOT freed by GC directly -- freed by Cleaner